MicroROM-NPU
Architecture Overview

Revision 0.2 | Aligned with the current 48-bit microroutine design

Document status
Current architecture proposal. Routine 0 (fetch/decode/dispatch), Routine 1 (LB), and Routine 2 (LW) have symbolic definitions; packed words have been reconstructed from the field layout but have not yet been verified by SystemVerilog simulation.

1. Purpose and current execution model

MicroROM-NPU is a standalone, microprogrammed execution engine for a deliberately selected subset of 32-bit RISC-V instructions. It is not an existing CPU core and is not a coprocessor attached to one. The project uses a two-dimensional vertical MicroROM, indexed by routine ID and local micro-step, to control architectural instruction execution and a small neural-inference workload.

Architecture item	Revision 0.2 decision
MicroROM organization	2D constant control store: 10 routine IDs (0-9) × 32 local steps (0-31) × 48 bits. Routine IDs 10-15 are invalid.
Microinstruction width	48 bits, with NEXT_ADDR[11:7] = 5-bit local step and CALL_MUL at bit 6.
Instruction scope	Eight 32-bit instruction encodings: LB, LW, SW, ADD, ADDI, BNE, BGE, JAL. Not full RV32I/RV32C compliance.
Architectural PC	Exactly one PC register. It advances during fetch; taken branch/jump routines may replace it in a later micro-step.
Memory behavior	Blocking request/response transactions. The sequencer holds the current micro-step until its operation completes.
ASIC direction	Constant ROM contents are initially intended to synthesize into standard-cell logic; no FPGA block-RAM inference is assumed.

2. Conceptual block architecture

Instruction memory request / response	IR + instruction decoder fields, immediates, validity, routine ID	2D MicroROM routine ID + local step -> 48-bit word
Data memory LB / LW / SW transactions	Datapath ALU, x0-x31 register file, internal bank	Microsequencer completion-gated transition logic
Internal state MAR, MDR, A, B, ACC, PROD, CTR, SIGN	Single PC advance / ALU target load / hold	Status illegal instruction, illegal microinstruction, memory fault

The diagram is a responsibility map, not a claim that all module boundaries or external port names are already frozen. The package microrom_pkg.sv contains constants and types; it is not a hardware module.

3. Microroutine map

Routine ID	Purpose	Status / notes
0	Instruction fetch, decode and dispatch	Symbolic sequence defined; instruction read uses MEM_CTRL=100. PC advances by 4 bytes after instruction capture in this version.
1	LB - load signed byte	Three steps: effective address, blocking byte read/sign extension, destination writeback.
2	LW - load word	Three steps: effective address, blocking word read, destination writeback. Initial version assumes aligned word addresses; unsupported access faults need defined handling.
3	SW - store word	Not yet formulated.
4	ADD	Not yet formulated.
5	ADDI	Not yet formulated.
6	BNE	Not yet formulated; branch target is MAR + B-immediate when taken.
7	BGE	Not yet formulated; signed comparison and target handling to be specified.
8	JAL	Not yet formulated; save advanced PC to RD, then load MAR + J-immediate into PC.
9	Repeated-add multiplication helper	Planned internal routine. Its caller/entry contract remains to be finalized; no selected architectural opcode dispatches directly to routine 9.

4. Routine 0 and memory wait contract

Routine 0 executes: Step 0 MAR <- PC; Step 1 instruction-read request and response wait; Step 2 IR <- MDR; Step 3 PC <- PC + 4; Step 4 decode/validate IR; Step 5 dispatch to routine 1-8 or enter the illegal-instruction error state.

Step	Completion condition	Sequencer action
0	MAR captures PC.	NEXT=SEQ advances to step 1.
1	Request is accepted, then response arrives and returned word is captured into MDR exactly once.	Hold at step 1 while pending; advance only after the entire read completes.
2	IR captures MDR.	Advance to step 3.
3	PC increment commits.	Advance to step 4.
4	Decoder finishes and sets instruction_valid and routine_id.	Advance to step 5.
5	A valid routine ID is ready.	DISPATCH enters routine ID at local step 0; invalid instructions enter the error state, not routine 0.

The memory controller must track whether the request was accepted so it cannot duplicate a read while awaiting a response. A response/fault protocol and exact timing are interface details still to be specified.

5. Architectural PC and register state

Routine 0 advances the sole PC after a successfully captured 32-bit instruction. MAR retains the original instruction address until another operation overwrites it. Branch and jump routines must calculate their target before reusing MAR for data access.

State	Purpose	Design rule
PC	Address of next instruction after fetch advancement.	One architectural PC; PC_CTRL selects hold, advance, or load ALU result.
MAR	Memory address register.	At fetch, captures the current PC; on LB/LW/SW it may be reused for a data address.
MDR	Memory data register.	Receives completed instruction/data read results. For instruction read, memory interface captures into MDR.
IR	Instruction register.	Holds the fetched 32-bit instruction; RS1/RS2/RD selector codes derive actual register indices from it.
x0-x31	Architectural 32-bit register file.	x0 reads zero and ignores writes. RS1, RS2, RD are dynamic selectors, not extra registers.
Internal bank	A, B, ACC, PROD, CTR, MAR, MDR, SIGN.	TMP is removed from current design; no IPC register is used.
Microsequencer state	routine_id[3:0], micro_step[4:0], and return state.	µPC is distinct from architectural PC. Local branches use 5-bit NEXT_ADDR.

Branch and jump target model

For the current 32-bit instruction subset, the original instruction address is retained in MAR. The ALU calculates MAR + sign-extended B-immediate for BNE/BGE or MAR + sign-extended J-immediate for JAL. PC_CTRL=10 loads the ALU result directly into PC. No IPC or TMP register is required. JAL first writes the already-advanced PC value to RD in a separate step.

6. Neural-inference workload and numeric contract

The target demonstration is a small single-neuron/dense-layer calculation, y = ReLU(sum(x[i] * w[i]) + b). Activations and weights are signed INT8; each exact product fits signed INT16; bias, accumulator, and output are signed INT32. The computation can be expressed as a program using the selected instruction subset and a repeated-add strategy, but the final calling/interface contract for internal routine 9 remains open.

Example: x = [2, 3, 1, 4], w = [1, 2, 3, 1], b = 2. Dot product = 15; biased sum = 17; ReLU output = 17. This is a functional demonstration, not a validated fault-detection product.

7. Verification gates before freezing the ROM

- Freeze the shared 48-bit package and all reserved/canonical field values.
- Validate all packed words with a field encoder/decoder test (round-trip each field).
- Test memory request backpressure, delayed response, same-cycle accept/response if supported, and fault response.
- Test PC hold/advance/target-load exclusivity; test branch target uses MAR before MAR is overwritten.
- Test x0 behavior, dynamic RS1/RS2/RD selection, signed LB extension, LW alignment behavior, and invalid instructions.
- Test routine ID / local step bounds, RESTART, RETURN, CALL_MUL valid/invalid cases, and illegal microinstruction side-effect suppression.

Revision 0.2 limits

Routine 0, LB and LW are specified at symbolic level; their packed words have been reconstructed from the updated fields. No RTL simulation, full instruction conformance, area, timing, or power result is claimed.

Freeze status summary

Item	Current status
48-bit field layout	Defined in Revision 0.2 format note; package/RTL validation pending.
Routine 0	Symbolic sequence defined; exact memory fault interface pending.
LB and LW	Three-step symbolic routines and field-packed words listed; not simulation-verified.
SW, ADD, ADDI, BNE, BGE, JAL	Routine tables remain to be formulated.
Routine 9 / CALL_MUL	Internal helper planned; caller convention remains open.